7d. Preliminaries on Types

Martin Alfaro
PhD in Economics

INTRODUCTION

High performance in Julia is intimately related to the notion of type stability. The definition of this concept is relatively straightforward: a function is type-stable when the types of its expressions can be inferred from the types of its arguments. When this property holds, Julia can specialize its computation method, resulting in fast code.

Despite its simplicity, type stability is subject to various nuances. In fact, a careful consideration of the property requires a solid foundation in two key areas: **Julia's type system and the inner workings of functions**. The current section equips you with the necessary knowledge to grasp the former, deferring the internals of functions to the next section. Moreover, **the explanations will focus on the case of scalars and vectors**, leaving more complex objects for subsequent sections.

Before you continue, I recommend reviewing the basics of types introduced here.

> ⚠️ **Warning!**
>
> **The subject is covered only to the necessary extent for understanding type stability**. Julia's type system is indeed quite vast, and a comprehensive exploration would warrant a dedicated chapter.

BASICS OF TYPES

Variables in Julia serve as mere labels for objects, with objects in turn holding values of specific types. The most common types for scalars are `Float64` and `Int64`, whose vector counterparts are `Vector{Float64}` and `Vector{Int64}`. Recall that `Vector` is an alias for a one-dimensional array, so that a type like `Vector{Float64}` is equivalent to `Array{Float,1}`.

> ❗ **`Int` As an Alternative to `Int64`**
>
> You'll notice that packages tend to use `Int` as the default type for integers. The type `Int` is an alias that adapts to your CPU's architecture. Since most modern computers are 64-bit systems, `Int` is equivalent to `Int64`. Instead, `Int` becomes `Int32` on 32-bit systems.

Julia's type system is organized in a hierarchical way. This feature permits the definition of subsets and supersets of types, which in the context of types are referred to as **subtypes** and **supertypes**. For instance, the type `Any` is a supertype that includes all possible types in Julia, thus occupying the highest position in the type hierarchy. Another example of supertype is `Number`, which encompasses all numeric types (`Float64`, `Float32`, `Int64`, etc.). [1]

Supertypes provide great flexibility for writing code. They enable the grouping of values under a common abstraction, making it possible to define operations generically. For instance, defining the `+` operator for the abstract type `Number` ensures its applicability to all numeric types, regardless of whether they are integers, floats, or their numerical precision.

A particular supertype known as `Union` will be instrumental for our examples. It allows variables to hold values of any type specified in its arguments. Its syntax is `Union{<type1>, <type2>, ...}`, so that a variable with type `Union{Int64, Float64}` could be either an `Int64` or `Float64`. Note that, by definition, union types are always supertypes of their constituent types.

ABSTRACT AND CONCRETE TYPES

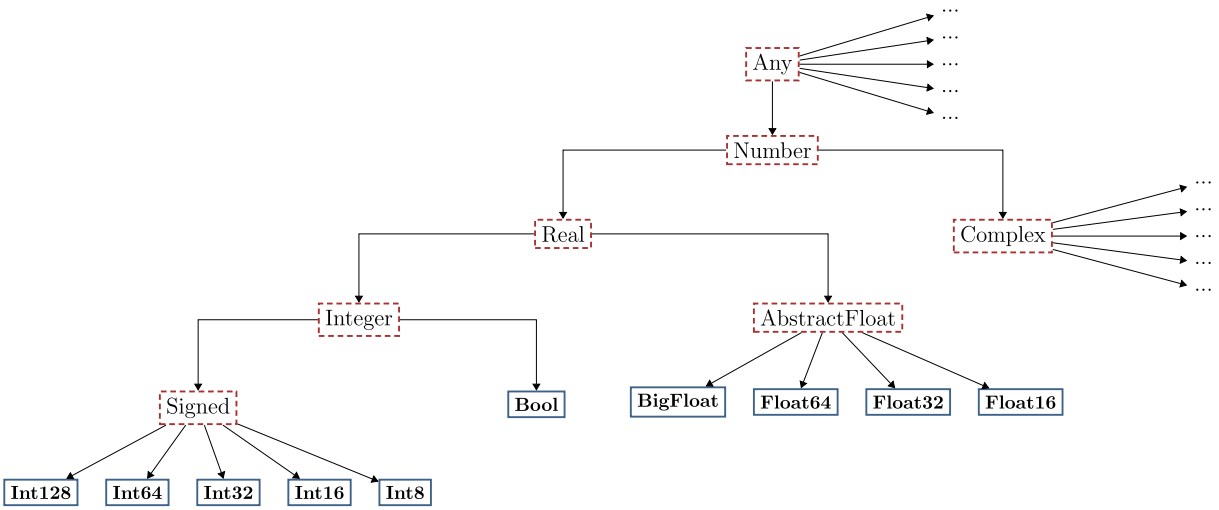
The hierarchical nature of types makes it possible to represent subtypes and supertypes through trees. Such a structure, in turn, gives rise to the notions of abstract and concrete types.

An **abstract type** acts as a parent category that groups related types, necessarily breaking down into subtypes. This means that it serves as a conceptual category, rather than a fully specified representation. The `Any` type in Julia is a prime example.

A **concrete type**, by contrast, is fully specified and has no subtypes. It represents an irreducible unit and therefore considered final, in the sense that it can't be further specialized within the hierarchy.

The diagram below illustrates the difference between abstract and concrete types for scalars. In particular, we present the hierarchy of the type `Number`. Note that the labels included in the diagram match the corresponding type name in Julia. [2]

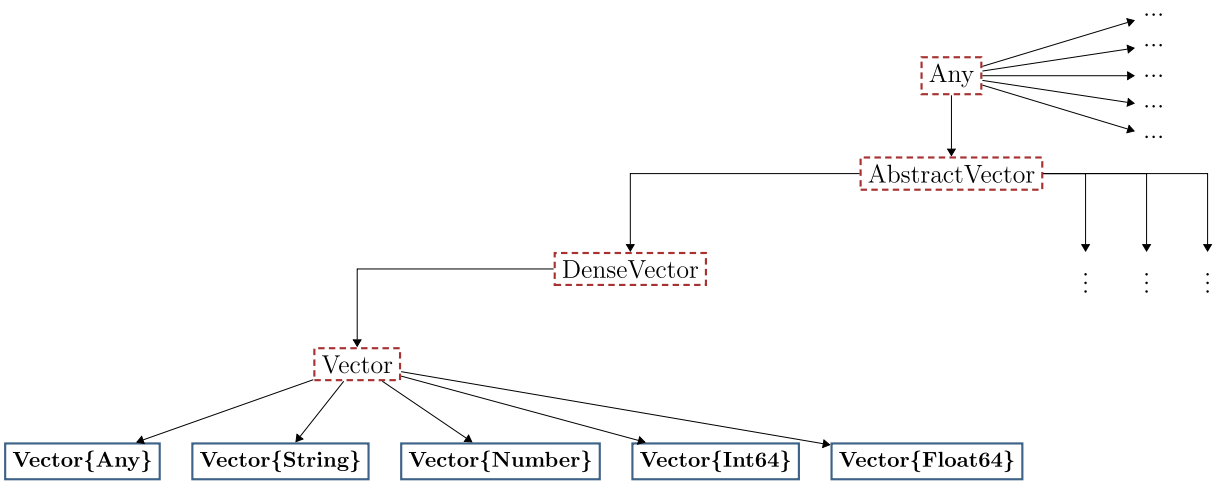
HIERARCHY OF TYPE NUMBER



Note: Dashed red borders indicate abstract types, while solid blue borders indicate concrete types.

The distinction between abstract and concrete types for scalars is relatively straightforward. Instead, their difference becomes more nuanced when considering vectors, as shown in the diagram below.

HIERARCHY OF TYPE VECTOR



Note: Dashed red borders indicate abstract types, while solid blue borders indicate concrete types.

The diagram reveals that `Vector{T}` **is a concrete type for each specific type** `T`. As a result, none of these concrete vector types have subtypes. This also means that a vector such as `Vector{Int64}` is not a subtype of `Vector{Any}`, even though `Int64` itself is a subtype of `Any` and `Any` is abstract.

The design is entirely consistent with the idea of vectors representing collections of homogeneous elements, where every element must share the same type.

ONLY CONCRETE TYPES CAN BE INSTANTIATED, ABSTRACT TYPES CAN'T

Instantiation refers to the act of creating a value of a specific type. A central principle of Julia's type system is that **only concrete types can be instantiated**. Abstract types, instead, describe sets of possible values, but never correspond to values themselves.

The distinction helps clarify the meaning of some widespread expressions used in Julia. For example, stating that a variable has type `Any` shouldn't be interpreted literally. Rather, it means the variable can hold values of any concrete type, considering that all concrete types are subtypes of `Any`.

The clarification is also important for what follows regarding type-annotation of variables. It implies that declaring a variable with an abstract type restricts the set of possible concrete types it can hold, even though the variable will ultimately adopt a concrete type.

RELEVANCE FOR TYPE STABILITY

At this point, you may be wondering how these concepts relate to type stability. The connection becomes clear when you consider how Julia performs computations.

High performance in Julia relies heavily on specializing the computation method. In the next section, we'll see that this specialization is unattainable in the global scope, as Julia treats global variables as potentially holding values of any type. In contrast, when code is written within a function, the execution process begins by determining the concrete types of each function argument. This information is then used to infer the concrete types of all the expressions in the function body.

When inference succeeds and all expressions have unambiguous concrete types, the function is considered **type stable**. This enables Julia to specialize its computation method and generate optimized machine code. If, instead, expressions could potentially take on multiple concrete types, performance is substantially degraded, as Julia must consider a separate implementation for each possible type.

For scalars and vectors, type stability essentially requires that expressions ultimately operate on **primitive types**. Examples of numeric primitive types include integers and floating-point numbers, such as `Int64`, `Float64`, and `Bool`. Thus, applying functions like `sum` to a `Vector{Int64}` or `Vector{Float64}` allows for full specialization, whereas applying them to a `Vector{Any}` prevents it.

> **String Objects**
>
> For text representation, the character type `Char` serves as a primitive type. Since a `String` is internally represented as a collection of `Char` elements, operations on `String` objects can also achieve type stability.

THE OPERATOR <: TO IDENTIFY SUPERTYPES

The remainder of this section is dedicated to operators and functions for handling types. Specifically, we'll introduce the operator `<:`, which checks whether one type is a subtype of another. Then, we'll examine strategies for constraining variables to specific types.

It's possible that you won't need to apply any of the techniques we present, as Julia automatically attempts to infer types when functions are called. Nonetheless, understanding these operators is essential for grasping upcoming material.

USE OF `<:`

The symbol `:<` tests whether a type `T` is a subtype of another type `S`. It can be used as an operator `T <: S` or as a function `<:(T,S)`. For example, `Int64 <: Number` and `<:(Int64, Number)` verify whether `Int64` is a subtype of `Number`, thus returning `true`. Below, we provide further examples.

ALL TRUE

```
# all the statements below are `true`
Float64 <: Any
Int64   <: Number
Int64   <: Int64
```

ALL FALSE

```
# all the statements below are `false`
Float64 <: Vector{Any}
Int64   <: Vector{Number}
Int64   <: Vector{Int64}
```

The fact that `Int64 <: Int64` evaluates to `true` illustrates a fundamental principle: **every type is a subtype of itself**. Moreover, in the case of concrete types, this is the only subtype.

THE KEYWORD `WHERE`

By combining `<:` with `Union`, you can also check whether a type belongs to a given set of types. For example, `Int64 <: Union{Int64, Float64}` assesses whether `Int64` equals `Int64` or `Float64`, thus returning `true`.

The approach can be made more widely applicable by using the `where` keyword with a type parameter `T`.[3] The syntax is `<type depending on T> where T <: <set of types>`. This entails that `T` covers multiple possible types.

CASE 1 - SCALARS

```
# all the statements below are `true`
Float64 <: Any
Int64   <: Union{Int64, Float64}
Int64   <: Union{T, String} where T <: Number       # `String` represents text
```

CASE 2 - VECTORS

```
# all the statements below are `true`
Vector{Float64} <: Vector{T} where T <: Any
Vector{Int64}   <: Vector{T} where T <: Union{Int64, Float64}
Vector{Number}  <: Vector{T} where T <: Any

# all the statements below are `false`
Vector{Float64} <: Vector{Any}
Vector{Int64}   <: Vector{Union{Int64, Float64}}
Vector{Number}  <: Vector{Any}
```

CASE 2 (EQUIVALENT)

```
# all the statements below are `true`
Vector{Float64} <: Vector{<:Any}
Vector{Int64}   <: Vector{<:Union{Int64, Float64}}
Vector{Number}  <: Vector{<:Any}

# all the statements below are `false`
Vector{Float64} <: Vector{Any}
Vector{Int64}   <: Vector{Union{Int64, Float64}}
Vector{Number}  <: Vector{Any}
```

Types relying on parameters like `T` are called **parametric types**. In the example above, these types allow us to distinguish between a concrete type like `Vector{Any}` and a set of concrete types `Vector{T} where T <: Any`, where the latter encompasses `Vector{Int64}`, `Vector{Float64}`, `Vector{String}`, etc.

> ⚠ **Warning!** - The Type `Any`
>
> When `<:` is omitted and we simply write `where T`, the statement is interpreted by Julia as `where T <: Any`. This is why the following equivalences hold.
>
> **EXAMPLE**
>
> ```
> # all the statements below are `true`
> Float64       <: Any
> Float64        <: T where T <: Any               # identical to
> the line above
>
> Vector{Int64} <: Vector{T} where T <: Any
> ```

ALTERNATIVE (EQUIVALENT)

```
# all the statements below are `true`
Float64       <: Any
Float64        <: T where T                        # identical to
the line above

Vector{Int64} <: Vector{T} where T
```

TYPE-ANNOTATING VARIABLES

In the following, we present methods for **type-annotating variables**. The technique can be used either to assert a variable's type **during an assignment** or to restrict the types of **function arguments**.

There are two approaches to type-annotating variables. The first one relies on the binary operator `::` and its syntax is `x::<type>`. The second approach leverages the Boolean binary operator `<:`, combined with `::` and the keyword `where`. Its syntax is `x::T where T <: <type>`, where `T` can be replaced with other symbol.

Next, we illustrate both methods, separately considering type-annotations for assignments and function arguments.

ASSIGNMENTS

Let's start illustrating the approaches based on scalar assignments. Each tab below declares an identical type for `x` and for `y`.

:: FOR ASSIGNMENT

```
x::Int64              = 2      # only reassignments to `Int64` are possible

y::Number              = 2      # only reassignments to `Float64`, `Float32`, `Int64`, etc
are possible
```

```
julia> x = 2.5
ERROR: InexactError: Int64(2.5)
julia> y = 2.5
2.5
julia> y = "hello"
ERROR: MethodError: Cannot convert an object of type String to an object of type Number
```

<: FOR ASSIGNMENT

```
x::T where T <: Int64  = 2      # only reassignments to `Int64` are possible

y::T where T <: Number = 2      # only reassignments to `Float64`, `Float32`, `Int64`, etc
are possible
```

```
julia> x = 2.5
ERROR: InexactError: Int64(2.5)
julia> y = 2.5
2.5
julia> y = "hello"
ERROR: MethodError: Cannot convert an object of type String to an object of type Number
```

> ⚠ **Warning!** - Modifying Types
>
> Once `x` has been assigned a type, that type can't be changed within the same Julia session. The only way to reset its type is to start a new session.

The fact that `x` holds the same type across all tabs follows because `T <: Float64` can only represent `Float64`. More specifically, `Float64` is a concrete type, which by definition has no subtypes other than itself. Considering this, scalar types are usually asserted using `::` rather than `<:`.

While this behavior holds for scalars, it doesn't apply to vectors. Specifically, using `::` in combination with `Vector{Number}` establishes that the object will have `Vector{Number}` as its concrete type. Instead, `Vector{T} where T <: Number` indicates that the elements of the vector will adopt a concrete subtype of `Number`.

VECTORS AND TYPE ANY

```
# `x` will always be `Vector{Any}`
x::Vector{Any}                = [1,2,3]

# `y` will always be `Vector{Number}`
y::Vector{Number}             = [1,2,3]
```

```
julia> typeof(x)
Vector{Any} (alias for Array{Any, 1})
julia> typeof(y)
Vector{Number} (alias for Array{Number, 1})
```

VECTORS AND TYPE NUMBER

```
# `x` is Vector{Int64}, but could eventually be `Vector{Float64}`, `Vector{String}`, etc
x::Vector{T} where T <:Any     = [1,2,3]

# `y` is now Vector{Int64}, but could eventually be `Vector{Float64}`, `Vector{Int32}`, etc
y::Vector{T} where T <: Number = [1,2,3]
```

```
julia> typeof(x)
Vector{Int64} (alias for Array{Int64, 1})
julia> typeof(y)
Vector{Int64} (alias for Array{Int64, 1})
```

The principles outlined apply even when a variable isn't explicitly type-annotated. The reason is that **an assignment without `::` implicitly assigns the type `Any` to the variable**. For example, the statements `x = 2` and `x::Any = 2` are equivalent.

The same occurs when omitting `<:` from the expression `where T`, which implicitly takes `T <: Any`. Thus, for instance, `x = 2` is equivalent to `x::T where T = 2` or `x::T where T <: Any = 2`. Considering this, all variables listed below have their types constrained in a similar manner.

TYPE 'ANY' WITH OPERATOR ::

```
# all are equivalent
a      = 2
b::Any = 2
```

TYPE 'ANY' WITH OPERATOR <:

```
# all are equivalent
a                  = 2
b::T where T       = 2
c::T where T <: Any = 2
```

Once we recognize that variables default to the type `Any`, it becomes clear why they can be reassigned with values of different types. For instance, given `a = 1`, executing `a = "hello"` afterward is valid, since `a` is implicitly type-annotated with `Any`.

> ⚠ **Warning!** - One-liner Statements Using `where`
>
> Be careful with one-liner statements using `where`, especially when `where T` is shorthand for `where T <: Any`. These concise statements can easily lead to confusion, as demonstrated below.

<: AND ASSIGNMENT

```
a::T where T                  = 2     # this is not `T = 2`, it's
`a = 2`

a::T where {T}                 = 2     # slightly less confusing
notation
a::T where {T <: Any}          = 2     # slightly less confusing
notation
```

<: AND FUNCTION

```
foo(x::T) where T             = 2     # this is not `T = 2`, it's
`foo(x) = 2`

foo(x::T) where {T}            = 2     # slightly less confusing
notation
foo(x::T) where {T <: Any}     = 2     # slightly less confusing
notation
```

FUNCTIONS

Function arguments can also be type-annotated. This is illustrated below, where functions are restricted to integer inputs exclusively.

OPERATOR ::

```
function foo1(x::Int64, y::Int64)
    x + y
end
```

```
julia> foo1(1, 2)
3
julia> foo1(1.5, 2)
ERROR: MethodError: no method matching foo1(::Float64, ::Int64)
```

OPERATOR <:

```
function foo2(x::Vector{T}, y::Vector{T}) where T <: Int64
    x .+ y
end
```

```
julia> foo2([1,2], [3,4])
2-element Vector{Int64}:
 4
 6
julia> foo2([1,2], [3.0, 4.0])
ERROR: MethodError: no method matching foo2(::Vector{Int64}, ::Vector{Float64})
```

Note that when both function arguments are annotated with the same type parameter `T`, they're constrained to share exactly the same type. Also notice that types like `Int64` preclude the use of `Float64`, even for numbers like `3.0`. Considering both facts, greater flexibility can be achieved by introducing separate type parameters, annotating them with a common abstract type like `Number`.

SAME TYPE

```
function foo2(x::T, y::T) where T <: Number
    x + y
end
```

```
julia> foo2(1.5, 2.0)
3.5
julia> foo2(1.5, 2)
ERROR: MethodError: no method matching foo2(::Float64, ::Int64)
```

DIFFERENT TYPES

```
function foo3(x::T, y::S) where {T <: Number, S <: Number}
    x + y
end
```

```
julia> foo3(1.5, 2.0)
3.5
julia> foo3(1.5, 2)
3.5
```

The greatest flexibility is achieved when we don't type-annotate function arguments at all, as they'll implicitly default to `Any`. This can be observed below, where all tabs define identical functions. Ultimately, type-annotating function arguments is only needed to prevent invalid usage (e.g., to ensure that `log` isn't applied to a negative value).

"ANY" (IMPLICIT)

```
function foo(x, y)
    x + y
end
```

"ANY" WITH ::

```
function foo(x::Any, y::Any)
    x + y
end
```

"ANY" WITH <:

```
function foo(x::T, y::S) where {T <: Any, S <: Any}
    x + y
end
```

EQUIVALENT TO <:

```
function foo(x::T, y::S) where {T, S}
    x + y
end
```

DEFINING VARIABLES WITH CERTAIN TYPE

To conclude this section, we present an approach for converting values into a specific type. The approach makes use of the so-called **constructors**, which are functions that create new instances of a concrete type. They're useful for transforming a variable `x` into another type.

Constructors are implemented via functions of the form `Type(x)`, where `Type` should be replaced with the name of the type (e.g., `Vector{Float64}`). Like any other function, constructors also support broadcasting.

SCALAR

```
x = 1

y = Float64(x)
z = Bool(x)
```

```
julia> y
1.0
julia> z
true
```

VECTOR

```
x = [1, 2, 3]

y = Vector{Any}(x)
```

```
julia> y
3-element Vector{Any}:
 1
 2
 3
```

BROADCASTING

```
x = [1, 2, 3]

y = Float64.(x)
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

Remark

Parametric types can be used as constructors. Moreover, abstract types may still serve as constructors, despite that they can't be instantiated. In such cases, Julia will attempt to convert the object to a specific concrete type. Nonetheless, not all abstract types can be used for this purpose.

SCALARS

```
x = 1

y = Number(x)
```

```
julia> typeof(y)
Int64
```

VECTORS

```
x = [1, 2]

y = (Vector{T} where T)(x)
```

```
julia> typeof(y)
Vector{Int64}
```

UNFEASIBLE

```
x = 1

z = Any(x)
```

```
ERROR: MethodError: no constructors have been defined for Any
```

An alternative to transform `x`'s type into `T` is given by the function `convert(T,x)`. Note that this method only works when a valid conversion exists, such as when `Float64` can be translated into an equivalent `Int64` (e.g., `3.0`).

SCALAR

```
x = 1

y = convert(Float64, x)
z = convert(Bool, x)
```

```
julia> y
1.0
julia> z
true
```

VECTOR

```
x = [1, 2, 3]

y = convert(Vector{Any}, x)
```

```
julia> y
3-element Vector{Any}:
 1
 2
 3
```

BROADCASTING

```
x = [1, 2, 3]

y = convert.(Float64, x)
```

```
julia> y
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

FOOTNOTES

[1.] Types don't necessarily follow a subtype-supertype hierarchy. For example, `Float64` and `Vector{String}` exist independently, without a hierarchical relationship. This fact will become clearer when the concepts of abstract and concrete types are defined.

[2.] The `Signed` subtype of `Integers` allows for the representation of negative and positive integers. Julia also offers the type `Unsigned`, which only accepts positive integers and comprises subtypes such as `UInt64` and `UInt32`.

[3.] `T` can be replaced by any other letter